

INSIGHTFORGE AI

Autonomous Business Analytics & Decision Intelligence Platform

69-Phase Complete Master Implementation Specification

MASTER DIRECTIVE

You are the **Lead Architect, Senior Data Engineer, Data Analyst, ML Engineer, AI Engineer, BI Developer, QA Engineer, DevOps Engineer, and Technical Project Manager** for this project.

Your responsibility is to design, implement, integrate, test, document, and polish the complete:

INSIGHTFORGE AI

Autonomous Business Analytics & Decision Intelligence Platform

You must treat this document as the **single source of truth** for the project.

The project must be built as a real working system.

Do not create:

- fake dashboards
- static results
- placeholder buttons
- fake AI responses
- disconnected scripts
- hard-coded analytical outputs
- mock integrations presented as real integrations

Every CORE feature must actually work.

CORE CONCEPT

InsightForge AI automatically converts raw business data into actionable intelligence.

The complete workflow is:

DATA



DETECT



VALIDATE



CLEAN



TRANSFORM



STORE



ANALYZE



DETECT CHANGE



DETECT ANOMALY



DETECT DRIFT



FIND ROOT CAUSE



QUANTIFY IMPACT



FORECAST



RECOMMEND



EXPLAIN WITH AI



VISUALIZE



REPORT



ALERT



AUDIT

Core intelligence philosophy:

DETECT → EXPLAIN → QUANTIFY → PREDICT → RECOMMEND → COMMUNICATE

TECHNOLOGY STACK

Use:

Data

- Python
- Pandas
- NumPy
- Alteryx

Database

- PostgreSQL
- pgAdmin

Analytics

- SQL
- Pandas
- SciPy
- Scikit-learn

ML

- Z-score
- IQR

- rolling baseline
- Isolation Forest
- exponential smoothing

BI

- Power BI
- DAX

Application

- Streamlit

AI

- LLM API
- controlled AI Analyst
- evidence-grounded analytics
- NL-to-SQL

Reporting

- ReportLab
- openpyxl

Automation

- Python watchdog/file watcher
- scheduler

Version Control

- Git
- GitHub

Optional

- Docker
- Docker Compose
- Airflow

Optional components must never delay the core system.

DOMAIN

Primary domain:

E-commerce / Retail

The platform should later be extensible to other industries, but Version 1 must focus on retail.

TARGET USERS

1. Business Manager
 2. Sales Manager
 3. Operations Manager
 4. Finance Manager
 5. Data Analyst
-

PHASE 0 — PROJECT INITIALIZATION

Objective

Create a clean and maintainable repository.

Tasks

Create:

InsightForge-AI/

Directories:

data/

src/

sql/

alteryx/

dashboard/

streamlit/

reports/

logs/

tests/

config/

scripts/

docs/

models/

Create:

README.md

requirements.txt

.env.example

.gitignore

run_pipeline.py

Acceptance Criteria

- Repository initializes.
- Python environment works.
- Git works.
- Secrets are excluded.

PHASE 1 — PROJECT ARCHITECTURE

Create complete architecture documentation.

Files:

docs/architecture.md

docs/system-components.md

docs/data-flow.md

docs/security.md

Document:

- components
- responsibilities
- dependencies

- communication
- failure paths

Acceptance Criteria

Architecture is understandable without reading source code.

PHASE 2 — REQUIREMENTS ENGINEERING

Define:

Functional requirements

Examples:

- ingest data
- validate schema
- clean records
- calculate KPIs
- detect anomalies
- identify root causes
- generate reports

Non-functional requirements

- reliability
- scalability
- security
- auditability
- performance
- maintainability

Create:

docs/requirements.md

PHASE 3 — BUSINESS QUESTIONS

Define questions the system must answer.

Examples:

What happened?

Why did revenue fall?

Which region caused the decline?

Which products are underperforming?

Which customers are at risk?

What is the estimated financial impact?

What will happen next month?

What should management investigate?

Map each question to data and analytics.

PHASE 4 — DATASET DESIGN

Design the retail dataset.

Required fields:

Order_ID

Order_Date

Customer_ID

Customer_Name

Customer_Segment

Product_ID

Product_Name

Category
Sub_Category
Region
State
City
Quantity
Unit_Price
Discount
Revenue
Cost
Profit
Payment_Method
Shipping_Days
Order_Status
Return_Status
Document every field.

PHASE 5 — DATA GENERATOR

Create:

scripts/generate_dataset.py

Generate at least:

150,000 records.

Use a fixed random seed.

Create realistic:

- pricing
- customer behavior
- seasonality
- product behavior

- geography
 - returns
 - shipping
-

PHASE 6 — BUSINESS DATA MODEL

Define business relationships.

Revenue:

$\text{Quantity} \times \text{Unit Price} \times (1 - \text{Discount})$

Profit:

$\text{Revenue} - \text{Cost}$

Ensure data consistency.

PHASE 7 — CUSTOMER DATA

Generate realistic customers.

Include:

- customer segment
- repeat customers
- new customers
- high-value customers
- low-value customers

Create realistic customer purchase patterns.

PHASE 8 — PRODUCT DATA

Generate:

- categories
- subcategories
- products
- product pricing

- cost
- product popularity

Make products behave differently.

PHASE 9 — GEOGRAPHICAL DATA

Use Indian geography.

Include:

- regions
- states
- cities

Maintain valid region-state-city relationships.

PHASE 10 — SEASONALITY ENGINE

Inject:

- weekends
- month-end effects
- festive periods
- seasonal demand

Document the generation logic.

PHASE 11 — DATA QUALITY INJECTION

Intentionally introduce:

- NULL values
- duplicates
- invalid dates
- negative quantities
- invalid discounts
- invalid customer IDs

- invalid categories
- extreme prices
- inconsistent text

These issues must be controlled and reproducible.

PHASE 12 — BUSINESS ANOMALY GENERATION

Inject a known anomaly.

Example:

West

→ Electronics

→ Laptop category

During a selected period:

Orders ↓

Revenue ↓

Returns ↑

Shipping Days ↑

Discount ↑

Document the ground truth.

This will be used to validate anomaly detection and RCA.

PHASE 13 — DAILY DATA GENERATION

Split dataset into daily files.

Example:

sales_2026_08_01.csv

sales_2026_08_02.csv

...

sales_2026_09_08.csv

Create:

data/incoming

data/raw

data/processed

data/rejected

data/archive

PHASE 14 — POSTGRESQL INSTALLATION & CONFIGURATION

Set up PostgreSQL.

Create:

database: insightforge

Configure:

- host
- port
- username
- password
- database

Use .env.

Never hard-code credentials.

PHASE 15 — DATABASE STAR SCHEMA

Create:

fact_sales

dim_date

dim_customer

dim_product

dim_region

Document relationships.

PHASE 16 — OPERATIONAL DATABASE TABLES

Create:

pipeline_runs

file_registry

data_quality_results

rejected_records

anomalies

recommendations

forecast_results

PHASE 17 — DATABASE INDEXING

Create indexes on:

- Order_ID
- Order_Date
- Customer_ID
- Product_ID
- Region
- Category

Optimize common analytical queries.

PHASE 18 — DATABASE CONNECTION LAYER

Create:

src/database.py

Implement:

- connection management
- transactions
- retries
- error handling

- safe query execution
-

PHASE 19 — FILE INGESTION

Create:

src/ingestion.py

Implement:

- file watcher
 - file validation
 - metadata collection
 - pipeline run ID
-

PHASE 20 — SHA-256 FILE FINGERPRINTING

Generate SHA-256 hash for each incoming file.

Use hash to detect duplicate files.

Store:

file_hash

in file_registry.

PHASE 21 — DATA CONTRACT

Create:

config/data_contract.yaml

Define:

- required columns
- types
- ranges
- accepted categories
- nullability
- uniqueness

PHASE 22 — SCHEMA VALIDATION

Implement:

src/validation.py

Check:

- missing columns
- extra columns
- wrong data types
- invalid values
- missing mandatory fields

Reject invalid schema.

PHASE 23 — ALTERYX INGESTION WORKFLOW

Create:

alteryx/01_ingestion.yxmd

Use actual Alteryx components.

Flow:

Input

→ Select

→ Clean

→ Validate

→ Output

PHASE 24 — ALTERYX DATA QUALITY WORKFLOW

Create:

alteryx/02_data_quality.yxmd

Implement:

- null detection

- duplicate detection
 - range checks
 - categorical validation
 - consistency checks
-

PHASE 25 — ALTERYX SALES ETL

Create:

alteryx/03_sales_etl.yxmd

Implement:

- transformation
 - joins
 - formulas
 - calculated fields
 - dimensional preparation
 - PostgreSQL output
-

PHASE 26 — ALTERYX CUSTOMER & PRODUCT ETL

Create:

alteryx/04_customer_product_etl.yxmd

Prepare:

- customer dimension
 - product dimension
 - segmentation fields
-

PHASE 27 — DATA QUALITY ENGINE

Create:

src/data_quality.py

Calculate:

Completeness

Validity

Uniqueness

Consistency

Accuracy

Timeliness

Referential Integrity

PHASE 28 — DATA QUALITY SCORE

Generate overall score.

Example:

97.6%

Rules:

$\geq 95\%$ PASS

90–94.99% WARNING

$< 90\%$ REJECT

Store historical quality scores.

PHASE 29 — REJECTED RECORD MANAGEMENT

Invalid records must be:

- stored
- classified
- traceable
- linked to pipeline run

Never silently delete invalid data.

PHASE 30 — CORE SQL KPI ENGINE

Create:

sql/kpi_queries.sql

Calculate:

- Revenue
 - Profit
 - Margin
 - Orders
 - Customers
 - Units
 - AOV
 - Return Rate
 - Average Discount
 - Shipping Time
-

PHASE 31 — ADVANCED SQL ANALYTICS

Demonstrate:

- JOIN
- CASE
- CTE
- subqueries
- window functions
- RANK
- DENSE_RANK
- LAG
- LEAD
- rolling average
- running total

PHASE 32 — ANALYTICAL VIEWS

Create PostgreSQL views for:

daily_kpis

monthly_kpis

regional_performance

category_performance

product_performance

customer_performance

PHASE 33 — PERIOD COMPARISON ENGINE

Compare:

day vs previous day

week vs previous week

month vs previous month

Generate percentage changes.

PHASE 34 — BUSINESS CHANGE DETECTION

Create:

src/change_detection.py

Detect significant movements.

Example:

Revenue ↓18.4%

Profit ↓23.7%

Orders ↓11.2%

Returns ↑8.3%

PHASE 35 — Z-SCORE ANOMALY DETECTION

Implement Z-score detection.

Store:

- metric
 - date
 - score
 - threshold
 - anomaly status
-

PHASE 36 — IQR ANOMALY DETECTION

Implement IQR.

Use:

Q1

Q3

IQR

Lower Bound

Upper Bound

Detect outliers.

PHASE 37 — ROLLING BASELINE

Create rolling:

- mean
- median
- standard deviation

Compare current values against expected range.

PHASE 38 — ISOLATION FOREST

Implement:

sklearn IsolationForest

Use appropriate features.

Return:

- anomaly score
 - prediction
 - confidence
-

PHASE 39 — ANOMALY FUSION

Combine:

Z-score

IQR

Rolling baseline

Isolation Forest

Generate a unified anomaly result.

PHASE 40 — SEVERITY ENGINE

Classify:

LOW

MEDIUM

HIGH

CRITICAL

Use:

- percentage deviation
 - business impact
 - confidence
 - persistence
-

PHASE 41 — DATA DRIFT DETECTION

Create:

src/drift_detection.py

Monitor:

- price distribution
 - quantity distribution
 - discount distribution
 - shipping distribution
 - category mix
 - region mix
-

PHASE 42 — DRIFT REPORTING

Generate:

Normal

Warning

Drift Detected

Store historical drift results.

PHASE 43 — ROOT-CAUSE ENGINE

Create:

src/root_cause.py

Drill down:

Region

→ Category

→ Subcategory

→ Product

→ Customer Segment

PHASE 44 — CONTRIBUTION ANALYSIS

Calculate contribution to KPI changes.

Example:

Revenue decline = ₹8.7L

West -₹5.1L

South -₹2.0L

North -₹1.1L

Other -₹0.5L

PHASE 45 — ROOT-CAUSE CONFIDENCE

Every RCA result should contain:

Metric

Change

Primary Driver

Evidence

Contribution

Confidence

PHASE 46 — BUSINESS IMPACT ENGINE

Create:

src/impact_analysis.py

Calculate:

Expected Revenue

Actual Revenue

Revenue Gap

Expected Profit

Actual Profit

Profit Gap

Revenue at Risk

Profit at Risk

Customers Affected

Orders Affected

PHASE 47 — CUSTOMER RFM ANALYSIS

Calculate:

Recency

Frequency

Monetary

Create:

Champions

Loyal

Potential Loyalists

New

At Risk

Lost

PHASE 48 — PRODUCT INTELLIGENCE

Classify products:

Star

High Profit

High Revenue

Fast Growing

Declining

High Return

Low Margin

Slow Moving

Create product health score.

PHASE 49 — FORECASTING

Create:

src/forecasting.py

Use:

Exponential Smoothing

Forecast:

- revenue
- profit
- orders

for:

- 7 days
 - 30 days
-

PHASE 50 — FORECAST EVALUATION

Measure forecasting quality.

Use appropriate metrics such as:

- MAE
- RMSE
- MAPE where appropriate

Document limitations.

PHASE 51 — RECOMMENDATION ENGINE

Create:

src/recommendations.py

Use transparent rules.

Example:

IF revenue declines

AND order volume declines

AND price is stable

THEN investigate demand or inventory.

PHASE 52 — PRIORITY ENGINE

Calculate:

Priority =

Severity × Impact × Confidence

Normalize to 0–100.

Classify:

LOW

MEDIUM

HIGH

CRITICAL

PHASE 53 — AI EVIDENCE LAYER

Before using an LLM, create a structured evidence package.

Example:

```
{  
  "metric": "revenue",  
  "current": 4370000,  
  "previous": 5240000,  
  "change_percent": -16.6,  
  "primary_region": "West",  
  "primary_category": "Electronics",  
  "impact": 870000  
}
```

The AI must consume verified evidence.

PHASE 54 — AI ANALYST

Create:

src/ai_analyst.py

Support:

Why did revenue decrease?

Which region performed worst?

Which products require attention?

What caused profit to decline?

What are the biggest risks?

Summarize this month.

The AI must not invent numbers.

PHASE 55 — AI EXPLANATION ENGINE

Generate structured explanations:

Summary

Evidence

Root Cause

Impact

Recommendation

Confidence

PHASE 56 — NATURAL LANGUAGE TO SQL

Implement:

Question

→ Intent

→ SQL

→ Validation

→ PostgreSQL

→ Result

→ Explanation

Only permit safe read queries.

Block:

DROP

DELETE

UPDATE

ALTER

TRUNCATE

INSERT

PHASE 57 — AI SECURITY

Implement:

- prompt injection protection
 - SQL validation
 - table whitelist
 - query timeout
 - result limits
 - secret protection
 - error sanitization
-

PHASE 58 — STREAMLIT APPLICATION

Create professional application.

Pages:

Overview

Pipeline

Data Quality

Anomalies

Root Cause

Business Impact

Forecast

Customers

Products

AI Analyst

Logs

PHASE 59 — STREAMLIT PIPELINE CONTROL

Display:

Pipeline Status

Last Run

Duration

Records

Quality

Anomalies

Failures

Provide controlled actions such as:

- run pipeline
- inspect run
- inspect errors

Do not create unsafe arbitrary execution controls.

PHASE 60 — POWER BI DATA MODEL

Connect Power BI to PostgreSQL/appropriate analytical outputs.

Create:

- relationships
- measures
- calculated metrics
- date hierarchy

Use DAX appropriately.

PHASE 61 — POWER BI EXECUTIVE DASHBOARD

Page:

Executive Overview

Display:

Revenue

Profit

Orders

Customers

AOV

Margin

Growth

Visuals:

- trends
 - regions
 - categories
 - products
-

PHASE 62 — POWER BI INTELLIGENCE PAGES

Create:

Business Drivers

- region
- category
- product
- contribution

Customer Intelligence

- RFM
- customer value
- segments

Product Intelligence

- growth
- margin
- returns

Risk & Anomalies

- severity
- anomaly timeline
- impact

PHASE 63 — POWER BI FORECAST & PIPELINE PAGES

Create:

Forecast

- actual
- forecast
- confidence interval

Pipeline Health

- quality score

- processing time
- records
- rejected rows
- failed runs

PHASE 64 — AUTOMATED EXCEL REPORT

Create:

src/reporting.py

Generate:

InsightForge_Report_DATE.xlsx

Sheets:

Executive Summary

KPIs

Regions

Categories

Products

Customers

Anomalies

Root Causes

Impact

Recommendations

Forecast

Data Quality

PHASE 65 — AUTOMATED PDF REPORT

Generate:

InsightForge_Executive_Report_DATE.pdf

Include:

- executive summary
- KPI changes
- major anomalies
- root causes
- impact
- forecast
- recommendations

Keep it management-friendly.

PHASE 66 — ALERT & EMAIL ENGINE

Create:

src/alerts.py

Rules:

LOW

→ Dashboard

MEDIUM

→ Streamlit

HIGH

→ Email

CRITICAL

→ Immediate Email

Email must contain:

- issue
- evidence
- impact

- recommendation

Attach PDF/Excel where appropriate.

PHASE 67 — OBSERVABILITY & FAILURE RECOVERY

Implement:

- structured logging
- pipeline IDs
- execution duration
- retries
- failure states
- error messages
- recovery states

Retry failed components up to three times where safe.

PHASE 68 — SECURITY, TESTING & PERFORMANCE

Security

Ensure:

- .env secrets
- no API keys in Git
- SQL injection protection
- safe LLM queries
- sanitized logs

Testing

Test:

Valid file

Missing columns

Extra columns

Duplicate file

Duplicate rows

NULLs

Invalid date

Negative quantity

Invalid discount

Unknown category

Empty file

Database unavailable

Email failure

Anomaly

No anomaly

Drift

RCA

Forecast

AI

NL-to-SQL

Performance

Measure:

- ingestion time
- ETL time
- database load time
- SQL time
- analytics time
- report generation time
- total pipeline time

PHASE 69 — FINAL END-TO-END DELIVERY

This is the final acceptance phase.

Run the entire system using a NEW dataset file.

Place:

sales_2026_09_09.csv

into:

data/incoming/

Then demonstrate:

File Detected



SHA-256 Generated



Duplicate Check



Schema Validation



Alteryx ETL



Data Quality



Rejected Records



PostgreSQL



SQL KPIs



Change Detection



Anomaly Detection



Drift Detection



Root Cause



Business Impact



Forecast



Recommendations



AI Explanation



Power BI



Streamlit



PDF



Excel



Email Alert



Audit Trail

FINAL SUCCESS CRITERIA

The project is complete only when:

DATA

- 150K+ records

- realistic relationships
- daily files
- controlled bad data
- controlled anomaly

DATA ENGINEERING

- automatic ingestion
- SHA-256 fingerprint
- duplicate detection
- data contracts
- Alteryx ETL
- quality scoring
- rejected records

DATABASE

- PostgreSQL
- star schema
- analytical views
- indexes
- audit tables

ANALYTICS

- KPIs
- advanced SQL
- period comparison
- customer analytics
- product analytics
- RFM

ML

- Z-score
- IQR

- rolling baseline
- Isolation Forest
- forecasting
- forecast evaluation

INTELLIGENCE

- change detection
- anomaly fusion
- severity
- drift
- RCA
- contribution analysis
- impact analysis
- recommendations
- priority scoring

AI

- evidence layer
- AI Analyst
- explanation
- NL-to-SQL
- SQL safety
- prompt injection protection

BI

- Power BI
- DAX
- executive dashboard
- business drivers
- customer intelligence
- product intelligence

- risk
- forecast
- pipeline health

AUTOMATION

- automated reports
- PDF
- Excel
- email
- alert severity

RELIABILITY

- logs
- audit trail
- retries
- failure recovery
- testing
- performance measurements

7-DAY EXECUTION STRATEGY

Although there are 69 implementation phases, do not interpret them as 69 separate days.

Group them into development waves.

DAY 1

Phases:

0–13

Focus:

- architecture
- requirements
- dataset
- data generation

- daily files
-

DAY 2

Phases:

14–29

Focus:

- PostgreSQL
 - schema
 - ingestion
 - validation
 - Alteryx
 - data quality
-

DAY 3

Phases:

30–34

Focus:

- SQL
 - KPIs
 - analytical views
 - change detection
-

DAY 4

Phases:

35–46

Focus:

- anomaly detection
- ML

- drift
 - RCA
 - contribution
 - impact
-

DAY 5

Phases:

47–52

Focus:

- RFM
 - product intelligence
 - forecasting
 - recommendations
 - priority
-

DAY 6

Phases:

53–67

Focus:

- AI
 - NL-to-SQL
 - Streamlit
 - Power BI
 - reporting
 - email
 - observability
-

DAY 7

Phase:

68–69

Focus:

- testing
 - security
 - performance
 - integration
 - documentation
 - demo
 - GitHub
 - final polish
-

TEAM SPLIT

PERSON 1 — DATA ENGINEERING & AUTOMATION

Own:

Dataset

Ingestion

Alteryx

Validation

Data Quality

PostgreSQL

ETL

Logging

Pipeline

Reports

Email

Failure Recovery

PERSON 2 — ANALYTICS & INTELLIGENCE

Own:

SQL Analytics

KPIs

Power BI

Anomaly Detection

Drift

RCA

Impact

Forecast

RFM

Product Intelligence

Recommendations

AI Analyst

SHARED

Integration

Testing

Git

Documentation

Demo

Presentation

README

DEVELOPMENT RULES

Rule 1

Never fake functionality.

Rule 2

Never hard-code analytical results.

Rule 3

PostgreSQL and deterministic analytics are the source of truth.

Rule 4

The LLM is an explanation/interface layer, not the source of numerical truth.

Rule 5

Raw data must remain immutable.

Rule 6

Invalid data must be traceable.

Rule 7

Every important pipeline stage must be logged.

Rule 8

Every important feature must have tests.

Rule 9

Never commit credentials.

Rule 10

Do not add complexity simply to make the architecture look advanced.

Rule 11

Finish P0 features before P1/P2 features.

Rule 12

Do not proceed to the next phase if the current phase has unresolved critical failures.

P0 FEATURES — ABSOLUTELY REQUIRED

These must work:

Ingestion

Alteryx ETL

Data Quality

PostgreSQL

SQL Analytics

Power BI

Automation

Anomaly Detection

Root Cause

Business Impact

Recommendations

Reports

Email

Logging

Testing

P1 FEATURES

Implement if P0 is stable:

Forecasting

RFM

Product Intelligence

Data Drift

Streamlit

P2 FEATURES

Implement only after P0/P1:

AI Analyst

NL-to-SQL

Agentic orchestration

Docker

Airflow

FINAL KILLER DEMO

The final demonstration should begin with:

"Imagine I am a data analyst receiving today's sales file."

Place:

sales_2026_09_09.csv

into:

data/incoming/

Do not manually execute individual processing stages.

Show:

✓ File detected

✓ File fingerprint generated

✓ Duplicate check passed

✓ Schema validated

✓ Alteryx workflow completed

✓ Data quality score: 97.6%

✓ 10,000 records received

✓ 9,781 valid

✓ 219 rejected

✓ PostgreSQL updated

✓ KPIs recalculated

⚠ Revenue anomaly detected

Revenue ↓18.4%

✓ Data drift checked

✓ Root cause identified

West

└─ Electronics

└─ Laptop X

✓ Business impact estimated

Revenue impact: ₹8.7L

✓ Forecast generated

✓ Recommendations generated

✓ AI explanation generated

✓ Power BI updated

✓ Excel generated

✓ PDF generated

✓ Alert sent

✓ Pipeline audit completed

Then ask the AI Analyst:

Why did revenue decrease?

The answer must be based on the verified analytical evidence.

FINAL PRODUCT STATEMENT

InsightForge AI should ultimately be described as:

An autonomous business analytics and decision-intelligence platform that transforms raw business data into validated insights, detects meaningful changes and anomalies, identifies likely root causes, quantifies business impact, forecasts future performance, generates evidence-based recommendations, and automatically communicates the results through dashboards, reports, and alerts.

FINAL PORTFOLIO DESCRIPTION

Use this on GitHub/resume:

Built InsightForge AI, an autonomous business analytics and decision-intelligence platform using Alteryx, PostgreSQL, SQL, Python, machine learning, and Power BI. Automated the complete analytics lifecycle from data ingestion and quality validation to KPI analysis, anomaly detection, root-cause analysis, business-impact estimation, forecasting, recommendations, automated reporting, and stakeholder alerts, with an evidence-grounded AI Analyst for natural-language business analysis.

FINAL INSTRUCTION TO CLAUDE

Start at **Phase 0**.

Do not attempt to generate the entire project blindly in one response.

Work incrementally.

For every phase:

1. Inspect the current project.
2. Determine what already exists.
3. Implement only what the phase requires.
4. Integrate with existing components.
5. Run tests.
6. Verify the feature.
7. Update documentation.
8. Report completion.
9. Identify any blockers.
10. Move to the next phase only when the current phase is stable.

At every stage, prefer:

working + tested + maintainable

over:

complex + impressive-looking + unfinished.

The final goal is not a collection of 69 features.

The final goal is **ONE cohesive, automated, reliable, demonstrable product: InsightForge AI.**